From Claim 7.9 and $y \sim_{[E'_q]} v$, it follows by Property 4.7(2) that $y \sim_{[E'_q]} w'_q.w_R$. Furthermore, from $E'_q \vdash q \diamond w'_q$ (which follows from $E'_q \vdash q \diamond w'_q.w_q$) and $E'_q \nvdash q \diamond y$, it follows that y is not a prefix of w'_q. Let u be the longest common prefix of y and $w'_q.w_R$. We claim that w'_q is a strict prefix of u. Otherwise, there are different processes p, p' and a sequence v'' such that $u.p'.v'' = w'_q$ and $u.p$ is a prefix of y. From $y \sim_{[E'_q]} w'_q.w_R$ and Property 4.7(2), we infer $y \sim_{[E'_q]} u.p'$. If $u.p' \in wut(E'_q)$ when y is inserted, we infer by Lemma 6.2 and Property 7.3(2) that $E'_q.y \propto E'_q.u.p'$. If $u.p' \notin wut(E'_q)$ when y is inserted, we also infer $E'_q.y \propto E'_q.u.p'$, since then y will be explored before $u.p'$. Thus $E'_q.y \propto E'_q.u.p'$, which implies $E'_q.u.p \propto E'_q.u.p'.v''$, which by handling of sleep sets implies $p \notin I_{[E'_q.u]}(p'.v'')$. By $y \sim_{[E'_q]} w'_q$, implying $u.p \sim_{[E'_q]} u.p'.v''$, we have $p \sim_{[E'_q.u]} p'.v''$, which is the same as $p \in WI_{[E'_q.u]}(p'.v'')$. By $p \notin I_{[E'_q.u]}(p'.v'')$ this implies $E'_q.u \vdash p \diamond (p'.v'')$. This implies that $p \in sleep(E'_q.w'_q)$, i.e., $p \in sleep(E)$. Hence by the construction of w_R, we have $p \notin WI_{[E'_q.u.p'.v'']}(w_R)$, which together with $E'_q.u \vdash p \diamond (p'.v'')$ implies $p \notin WI_{[E'_q.u]}(p'.v''.w_R)$, which implies $u.p \not\sim_{[E'_q]} u.p'.v''.w_R$, which implies $y \not\sim_{[E'_q]} w'_q.w_R$, which contradicts the construction of y.

Thus, w'_q is a strict prefix of u. Since $E'_q.y$, and hence $E'_q.u$, is explored by the algorithm, we have $E'_q.u \propto E'_q.w'_q$. Moreover, since u is a prefix of $w'_q.w_R$, we infer that $E'_q.u$ is a prefix of $E'_q.w'_q.w$. This means that there is a sequence w'' such that $E'_q.u.w'' \simeq E.w$. It follows by the inductive hypothesis applied to $E'_q.u$ that the algorithm has explored some maximal sequence in $[E'_q.u.w'']_{\simeq}$ and hence in $[E.w]_{\simeq}$. This contradicts the assumption at the beginning of the inductive step. This concludes the proof of the inductive step, and Theorem 7.4 is proven. □

7.3 Optimality

In this section, we prove that optimal-DPOR is optimal in the sense that it never explores two different but equivalent execution sequences and never encounters sleep set blocking. The following theorem, which is essentially the same as Theorem 3.2 of Godefroid et al. [1995], establishes that sleep sets alone are sufficient to prevent exploration of two equivalent *maximal* execution sequences.

Theorem 7.11. *Optimal-DPOR never explores two maximal execution sequences which are equivalent.*

Proof. Assume that E_1 and E_2 are two equivalent maximal execution sequences that are explored by the algorithm. Then they are both in $\mathcal{E}$. Assume w.l.o.g. that $E_1 \propto E_2$. Let E be their longest common prefix, and let $E_1 = E.p.v_1$ and $E_2 = E.v_2$. By Lemma 7.2 we have $p \notin I_{[E]}(v_2)$, which contradicts $E_1 \simeq E_2$ and the maximality of E_1 and E_2. □

We will now prove that Algorithm 2 is optimal in the sense that it never encounters sleep set blocking. Let us first define this precisely.

Definition 7.12 (Sleep Set Blocking). A call to *Explore*(E, *Sleep*, *WuT*) is *sleep set blocked* during the execution of Algorithm 2 if $enabled(s_{[E]}) \neq \emptyset$ and $enabled(s_{[E]}) \subseteq Sleep$.

Now let us state and prove the corresponding optimality theorem.

Theorem 7.13 (Optimality of Optimal-DPOR). *During any execution of Algorithm 2, no call to Explore*(E, *Sleep*, *WuT*) *is ever sleep set blocked.*

Proof. Consider a call *Explore*(E, *Sleep*, *WuT*) during the exploration. Then any sequence in *WuT* is enabled after E. By Lemma 7.1, *WuT* is a wakeup tree after $\langle E, Sleep \rangle$. Thus, if $Sleep \neq \emptyset$, then *WuT* contains a sequence w such that $Sleep \cap WI_{[E]}(w) = \emptyset$. Letting p be the first process in w, this implies $p \notin Sleep$, implying that p is enabled and thus $enabled(s_{[E]}) \nsubseteq Sleep$. □